HADES

Web Security Scanner
A Beginner's Guide to Every Module

For authorised security testing only. Scanning systems without explicit written permission is illegal. This guide explains what each module checks and why it matters, in plain language for people new to cybersecurity.

Generated 2026-06-04

How to read this guide

Hades runs many small **modules**. Each one checks one thing about a website. For every module below you will find: **what it really does and looks for**, **what an attacker could do if it finds a problem**, and a short **good-to-know** tip. You do not need any prior knowledge — terms are explained as they appear.

Severity levels

Level	Meaning
INFO	Just information / context. Never affects the score.
LOW	Minor hardening gap; little direct risk on its own.
MEDIUM	Worth fixing; helps attackers or leaks information.
HIGH	Serious; a likely path to compromise.
CRITICAL	Severe; often a direct way in (secrets, code execution).

1 · Reconnaissance Modules

basic_info

Target Fingerprint

What it really does & looks for: Makes a first request to the site and records the essentials. It reports the IP address the domain points to, the web-server software, the page load time, the page title, and guesses the operating system from a network value called TTL (a number that counts how many hops a packet can take).

If it finds something — the attack: On its own this is harmless reconnaissance, but it is the attacker's first step: knowing the server software and OS tells them which known weaknesses to try next.

Good to know: Everything here is INFO and never lowers your security score — it is context, not a problem.

whois_lookup

Domain Registration Lookup

What it really does & looks for: Queries public registration databases (WHOIS) for the domain. It returns who registered the domain (the registrar), when it was created, when it expires, and the name servers. It is smart enough to skip platform sub-domains like yoursite.vercel.app where WHOIS is meaningless.

If it finds something — the attack: Attackers use the expiry date (a domain about to expire can be hijacked) and registration details for social-engineering or phishing that looks legitimate.

Good to know: An expiry date that is very close is flagged because an expired domain can be taken over by anyone.

dns_check

Email Security Records (DNS)

What it really does & looks for: Looks up the domain's DNS records that protect email. It checks for SPF, DKIM, and DMARC (three records that prove an email truly came from your domain) and MX records (the mail servers).

If it finds something — the attack: If SPF/DMARC are missing, an attacker can send emails that look like they come from your domain — classic phishing and CEO-fraud. This is one of the most common real-world weaknesses.

Good to know: Missing SPF or DMARC does not break your website, but it lets criminals impersonate your email address.

ssl_check

TLS / Certificate Inspector

What it really does & looks for: Examines the HTTPS certificate that encrypts traffic. It reads the certificate issuer, the TLS protocol version, and how many days until the certificate expires; it also flags a site served over plain HTTP (no encryption at all).

If it finds something — the attack: No HTTPS means anyone on the same network (public Wi-Fi, a malicious router) can read or modify the traffic — passwords included. An expired certificate breaks trust and can hide an attacker.

Good to know: A certificate expiring soon is flagged early so it can be renewed before visitors see scary browser warnings.

port_scan

TCP Port Scanner

What it really does & looks for: Tries to open a network connection to common service ports on the server's IP. It checks ~26 ports (databases like MySQL/Redis/MongoDB, remote access like RDP/VNC/SSH, web/mail) and grabs a short 'banner' to confirm the service. It first probes random unused ports to detect an 'accept-all' host.

If it finds something — the attack: An exposed database or remote-desktop port is a direct doorway: attackers brute-force the login or exploit the service to take over the server. Databases should never be reachable from the internet.

> **Good to know:** If the host answers EVERY port (a firewall/honeypot trick), Hades reports one 'unreliable' note instead of dozens of fake 'open ports'. Behind a CDN, the scanned IP is the edge, not the real server.

waf_detect

Firewall / CDN Detection

What it really does & looks for: Inspects response headers and behaviour to spot a protective layer in front of the site. It identifies WAFs and CDNs such as Cloudflare, Akamai, AWS, or Fastly.

If it finds something — the attack: Knowing a WAF exists tells an attacker their payloads may be filtered, so they will craft bypass techniques. For a defender it confirms a protection layer is active.

> **Good to know:** A WAF is good news — but it also explains why other modules may get blocked (403s) during a scan.

tech_stack

Technology Fingerprint

What it really does & looks for: Reads headers, HTML, scripts, and cookies to identify the software powering the site. It detects the web server, programming language, frameworks, JavaScript libraries (jQuery, React…), CMS, and — crucially — their version numbers when visible.

If it finds something — the attack: Versions are gold for attackers: an old library with a published vulnerability (CVE) can often be exploited with a ready-made tool. This module feeds the CVE lookup.

> **Good to know:** Hiding version numbers (server tokens, X-Powered-By) makes an attacker's job harder.

2 · Website & Misconfiguration Modules

headers_check

HTTP Security Headers Audit

What it really does & looks for: Reads the response headers that browsers use to enforce security rules. It deeply checks Content-Security-Policy (parsed directive by directive), HSTS (forces HTTPS), X-Frame-Options, the cross-origin headers (COOP/COEP/CORP), and flags headers that leak software versions.

If it finds something — the attack: Missing headers remove browser-side defences: no CSP makes XSS far easier, no HSTS allows downgrade attacks, no X-Frame-Options enables clickjacking.

> **Good to know:** These are 'hardening' gaps — rarely exploitable alone, but they make every other attack easier. CSP missing is the highest-impact one.

robots_txt

robots.txt Analyzer

What it really does & looks for: Downloads /robots.txt and studies what it reveals. It classifies each disallowed path (admin, login, backups, .git…) with precise matching, then ACTIVELY visits each sensitive path to see if it is really reachable, and flags wildcard rules that leak file types.

If it finds something — the attack: robots.txt must list paths in clear text to hide them from search engines — which paradoxically hands attackers a map of your sensitive areas. A disallowed path that is also reachable is escalated.

> **Good to know:** A path here that returns 404 is a 'stale' leftover (low value); one that returns 200 is a real, reachable target.

sitemap

Sitemap Parser

What it really does & looks for: Finds and reads XML sitemaps (and any declared in robots.txt). It counts the listed URLs, follows sitemap indexes one level deep, and flags any sensitive URL the sitemap advertises (admin, staging, .git…).

If it finds something — the attack: A sitemap that lists internal or admin URLs gives attackers a curated list of interesting targets without any guessing.

> **Good to know:** Sitemaps are meant to be public; the only concern is when they expose URLs that should stay private.

cms_detect

CMS Detection

What it really does & looks for: Fingerprints the content-management system running the site. It identifies WordPress, Joomla, Drupal, and similar platforms (and their version where possible).

If it finds something — the attack: Each CMS has a huge catalogue of known plugin/theme vulnerabilities. Knowing the CMS and version lets an attacker pick a matching exploit immediately.

> **Good to know:** WordPress powers a large share of the web and is the most-attacked CMS, so detecting it matters.

admin_panel

Admin / Login Panel Finder

What it really does & looks for: Probes known admin paths and VERIFIES whether each is really a login or admin interface. Instead of trusting status codes, it checks the page content for a real login form, an authenticated admin UI, HTTP authentication, or a redirect to a login page.

If it finds something — the attack: An exposed admin panel is a prime brute-force and credential-stuffing target. A panel reachable without logging in can be an instant full compromise.

Good to know: Thanks to content verification, it no longer cries 'admin found' for every page on a catch-all site — only genuine login/admin pages are reported.

dir_scan

Directory & Path Brute-Forcer

What it really does & looks for: Requests a wordlist of common directories/paths to discover what exists on the server. It distinguishes open directory listings (High), normal accessible paths, auth-protected paths (401), forbidden paths (403), and redirects. It learns the server's 'not found' behaviour first.

If it finds something — the attack: Discovered admin areas, backups, or config folders give attackers new entry points. An open directory listing exposes every file in a folder.

Good to know: Smart baselines collapse the noise: if the server answers 200 (or 403, or 500) to everything, Hades reports one note instead of hundreds of fake hits.

subdomain_scan

Subdomain Enumeration & Takeover

What it really does & looks for: Discovers sub-domains both actively (DNS brute-force) and passively, then checks each for takeover. Active resolving is combined with Certificate Transparency logs (crt.sh) to reveal sub-domains that were never guessable. Each one is then probed for dangling-DNS takeover.

If it finds something — the attack: Forgotten sub-domains (dev, staging, old) widen the attack surface and are often less protected. A 'subdomain takeover' lets an attacker host their own content on YOUR sub-domain — perfect for phishing.

Good to know: A wildcard-DNS check prevents false positives where every name resolves to the same address.

broken_links

Broken Link Checker

What it really does & looks for: Checks the HTTP status of every link the crawler found and classifies it correctly. It separates genuinely dead links (404/410), server errors (5xx), access-controlled links (401/403), and rate-limiting (429).

If it finds something — the attack: A 404 is just broken usability/SEO. But a wall of 403s usually means a WAF is blocking the scanner — not broken links — so it is grouped into one advisory instead of dozens of false alarms.

Good to know: A 403 means 'forbidden', NOT 'broken' — the page likely works fine in a real browser.

http_methods

HTTP Methods Auditor

What it really does & looks for: Discovers which HTTP methods the server accepts and actively verifies the dangerous ones. For PUT it tries a safe upload of a unique harmless file and reads it back; for TRACE it checks for echo (XST); DELETE is reported as advertised-only (never tested, that would be destructive).

If it finds something — the attack: A confirmed PUT upload is critical: an attacker can place a web shell and run commands on the server. A real TRACE echo can be used to steal headers.

Good to know: It no longer trusts the 'Allow' header blindly — only a method proven to work is rated High/Critical. Active write tests are skipped in passive/safe mode.

backup_files

Backup & Source File Hunter

What it really does & looks for: Looks for backup or temporary copies derived from the target itself. It tries archive names based on the hostname (site.zip, site.tar.gz), common backup names, and editor backups of real pages (index.php~, config.php.bak, vim .swp files), confirming archives by their magic bytes.

If it finds something — the attack: A downloadable backup often contains the full source code, database dumps, and hard-coded passwords — effectively the keys to the kingdom in one file.

Good to know: It confirms a real archive by content, so an HTML 'not found' page is never mistaken for a backup.

sensitive_files

Sensitive File Exposure

What it really does & looks for: Probes well-known paths that expose secrets, configuration, or source control. It checks for .env files, .git repositories, cloud credentials (AWS/GCP), SSH keys, framework configs, database dumps, and more — with content checks to avoid false positives.

If it finds something — the attack: A readable .env or .git directory can leak database passwords, API keys, and the entire source code, leading directly to full compromise.

Good to know: If the server blocks every sensitive path (a good 'deny-all' rule), Hades reports one positive note instead of many false alarms.

cookie_analysis

Cookie Security Flags

What it really does & looks for: Inspects the cookies the site sets in the browser. It checks for the Secure flag (only sent over HTTPS), HttpOnly (hidden from JavaScript), and SameSite (limits cross-site sending), plus prefixes like __Host-.

If it finds something — the attack: A session cookie without HttpOnly can be stolen by XSS to hijack a user's session; without Secure it can leak over plain HTTP; without SameSite it enables CSRF.

Good to know: These flags are the difference between an XSS bug being annoying and being account-takeover.

redirect_chain

Redirect Chain Auditor

What it really does & looks for: Follows the chain of redirects starting from the target URL. It records every hop and flags an HTTPS-to-HTTP downgrade, redirects that leave your domain, missing HTTP-to-HTTPS upgrade, and chains that are too long.

If it finds something — the attack: A downgrade redirect exposes traffic to interception; an open/off-domain redirect can be abused in phishing to make a malicious link look like it starts on your trusted site.

Good to know: A clean single HTTP-to-HTTPS redirect is exactly what you want to see.

email_exposure

Exposed Email Finder

What it really does & looks for: Collects email addresses that appear across the crawled pages. It harvests addresses from mailto links and page text, separating on-domain addresses from third-party ones.

If it finds something — the attack: Published addresses fuel spam, phishing, and targeted social-engineering. An on-domain address is a direct spear-phishing target for staff.

Good to know: Use contact forms or obfuscation instead of publishing personal mailboxes in plain text.

favicon_hash

Favicon Fingerprint

What it really does & looks for: Computes a hash of the site's little browser-tab icon, the same way Shodan does. It produces a number you can search (http.favicon.hash:VALUE) to find other servers using the same icon, and recognises a few well-known default favicons.

If it finds something — the attack: Identical favicon hashes reveal shared frameworks, hidden admin panels, or related infrastructure — a quiet way to map an organisation's assets.

Good to know: A default framework favicon quietly tells the world what software you run; a custom icon avoids that.

cors_check

CORS Misconfiguration

What it really does & looks for: Sends requests with a crafted Origin header to test cross-origin sharing rules. It checks whether the server reflects any origin and allows credentials (the dangerous combination).

If it finds something — the attack: A permissive CORS policy lets a malicious website read authenticated responses from your site on behalf of a logged-in victim — leaking their private data.

Good to know: CORS bugs are subtle: 'Access-Control-Allow-Origin: *' is usually fine, but reflecting the caller's origin WITH credentials is the real danger.

clickjacking

Clickjacking Verdict

What it really does & looks for: Decides whether the page can be loaded inside an attacker's invisible frame, and how risky that is. It combines X-Frame-Options and CSP frame-ancestors into a single 'framable?' answer, and weights the risk by whether the page has a login form or other forms.

If it finds something — the attack: If framable, an attacker overlays an invisible copy of your page over a decoy and tricks users into clicking real buttons (changing settings, confirming actions) without realising it.

Good to know: A framable page with no interactive elements is low impact; one with a login form is high.

dir_listing

Open Directory Listing

What it really does & looks for: Detects folders whose entire contents are publicly browsable. It checks directories discovered by the crawler plus common upload/asset/backup folders for the tell-tale 'Index of /' auto-index page, and lists the exposed files.

If it finds something — the attack: An open listing hands attackers a full inventory of files — often including backups, uploads, or documents that were never meant to be public.

Good to know: It reuses the crawler's findings, so it tests folders that actually exist on the site rather than guessing blindly.

blacklist_check

Reputation / Blocklist Check

What it really does & looks for: Checks the site's IP and domain against public DNS blocklists (DNSBLs). Using the standard, key-free DNSBL protocol it queries lists like Spamhaus, SpamCop, and Barracuda.

If it finds something — the attack: Being listed signals a history of spam or malware — or a current compromise — and badly hurts email deliverability and reputation. It can be an early warning that a server is already hacked.

> **Good to know:** Behind a CDN the checked IP is the shared edge, which is rarely listed — so a clean result is less meaningful there.

screenshot

Homepage Screenshot

What it really does & looks for: Opens the homepage in a real headless browser and saves a picture of it. It captures a full-page PNG into the screenshots/ folder using Chromium.

If it finds something — the attack: Not an attack itself — it gives a human a quick visual of what the target looks like, useful for triage across many sites.

> **Good to know:** If the browser engine is not installed it fails gracefully with a hint, never breaking the scan.

3 · Vulnerability Modules

sqli_detect

SQL Injection Detection

What it really does & looks for: Injects test payloads into URL/form parameters to see if they reach a database query. It uses three techniques: error-based (triggering a database error message), boolean-based blind (a TRUE condition behaves like normal, a FALSE one differs), and time-based blind (a SLEEP payload makes the page hang for a measurable, scaling delay).

If it finds something — the attack: SQL injection lets an attacker read or modify the database directly — dumping all users and passwords, bypassing logins, sometimes taking over the server. It is one of the most damaging web vulnerabilities.

Good to know: It DETECTS the flaw; it does not dump data itself. On a confirmed hit Hades shows a ready sqlmap command and, with the --exploit flag, can launch sqlmap for you (authorised targets only). Skipped in safe mode.

xss_detect

Cross-Site Scripting (XSS) Detection

What it really does & looks for: Injects a marker into inputs and analyses exactly where and how it is reflected back. It first finds the reflection context (HTML text, an attribute, a JavaScript string, or a comment), then sends the minimal 'breakout' payload for that context and checks whether the special characters survive unencoded.

If it finds something — the attack: Reflected XSS lets an attacker run their JavaScript in a victim's browser: stealing session cookies, performing actions as the victim, or defacing the page. Combined with weak cookie flags it becomes account takeover.

Good to know: Context analysis makes results credible: a reflection whose special characters are encoded is reported only as Low (likely safe), while a real breakout is High.

command_injection

OS Command Injection

What it really does & looks for: Injects shell separators and commands (;id, | whoami, & dir, sleep) into parameters and form fields. It confirms a hit two ways: the command's OUTPUT appears in the page (uid=0(root), Volume Serial Number), or an injected sleep makes the response hang for a measurable delay that SCALES with the requested time.

If it finds something — the attack: Command injection is total compromise: the attacker runs arbitrary operating-system commands on the server — read or change any file, install a backdoor, pivot into the internal network. It is among the most severe web flaws.

Good to know: Verified, never guessed: a delay that scales (5s clearly longer than 2s) rules out a merely slow page. A ready commix command is attached. Skipped in safe mode.

ssti_detect

Server-Side Template Injection (SSTI)

What it really does & looks for: Injects a distinctive maths expression in every template syntax ({{7*7}}, ${7*7}, <%= 7*7 %>, #{7*7}…). It flags the input only if the server returned the COMPUTED result (e.g. the product) instead of the literal text — proof the value is evaluated as a template, not just echoed.

If it finds something — the attack: SSTI usually escalates to remote code execution: the attacker runs code inside the template engine to read secrets, files, or take over the server.

Good to know: The maths uses large numbers so a coincidental match is impossible, and the firing syntax fingerprints the engine (Jinja2, Twig, FreeMarker…). A tplmap command is attached.

lfi_detect

Local File Inclusion / Path Traversal

What it really does & looks for: Requests well-known system files through traversal payloads (../../../etc/passwd, ..\windows\win.ini, PHP filter wrappers). It confirms the read by the file's ACTUAL content (the root:x:0:0: line of /etc/passwd, the [fonts] section of win.ini, or base64 that decodes to PHP source).

If it finds something — the attack: LFI lets an attacker read arbitrary server files — configuration, source code, credentials — and can chain to remote code execution via log or session poisoning.

Good to know: Confirmed by file content, never by guesswork, so false positives are rare.

open_redirect

Open Redirect

What it really does & looks for: Injects a unique external URL into redirect-style parameters (url, next, return, redirect…). It requests the page without following redirects and flags when the server forwards the user to that external host (Location header, meta refresh, or JavaScript).

If it finds something — the attack: An open redirect makes a link that STARTS on your trusted domain land on the attacker's site — ideal for phishing — and can bypass redirect allowlists in login/OAuth flows.

Good to know: Severity rises to High when the parameter drives an authentication flow.

ssrf_detect

Server-Side Request Forgery (SSRF)

What it really does & looks for: Injects internal and cloud-metadata URLs (127.0.0.1, 169.254.169.254, file:///etc/passwd) into URL-shaped parameters. It flags responses that come back containing content only the SERVER could fetch — cloud metadata, or the contents of a local file.

If it finds something — the attack: SSRF makes the server send requests on the attacker's behalf: stealing cloud credentials from the metadata service, scanning the internal network, or reaching admin panels never exposed to the internet.

Good to know: In-band detection only; truly blind SSRF needs an out-of-band (OOB) test, planned as a future addition.

cve_mapping

Known Vulnerability (CVE) Lookup

What it really does & looks for: Takes the software versions found by tech_stack and asks the official NVD database for matching vulnerabilities. For each versioned component it retrieves the top CVEs (public vulnerability records) with their severity score (CVSS).

If it finds something — the attack: A matching CVE often means a ready-made exploit exists: an attacker just runs it against your outdated component. This turns 'old version' into 'known way in'.

Good to know: Set an NVD_API_KEY environment variable for a higher rate limit and more complete results.

default_creds

Default Credentials Advisory

What it really does & looks for: Fingerprints management interfaces that are famous for shipping with default passwords. It detects things like phpMyAdmin, Tomcat Manager, Jenkins, or Grafana and lists their documented default logins — WITHOUT ever trying to log in.

If it finds something — the attack: If an admin never changed the default password (admin/admin, tomcat/tomcat…), anyone who finds the panel walks straight in. This is an extremely common real-world breach cause.

> **Good to know:** By design it never submits credentials — actively trying passwords against someone else's system is intrusive and can be illegal. Verify manually on your own systems.

4 · Database Security Audit (db_scan)

db_security (db_scan)

Red-Team Database Security Audit

What it really does & looks for: A single dedicated module (run with --profile db_scan) that hunts database exposure end to end and then helps exploit it. It scans the common database ports and fingerprints the engine from its banner (MySQL, PostgreSQL, MSSQL, Oracle, MongoDB, Redis, Elasticsearch, CouchDB, Cassandra, Memcached); tests each for access WITHOUT a password and, when open, actually pulls proof-of-data — Redis keys plus a CONFIG check that means remote code execution, Elasticsearch index names and document counts, CouchDB database names; probes crawled parameters for SQL and NoSQL injection; and checks TLS on the database port.

If it finds something — the attack: An open, unauthenticated database is an instant full data breach: anyone on the internet reads or modifies everything. A confirmed SQL injection is dumpable, and an exposed Redis CONFIG turns into a server takeover (web shell / SSH key / cron).

Good to know: Confirmed SQL injection found here is exploitable with the same --exploit sqlmap launcher. Destructive checks (default creds, time-based SQLi, NoSQL) are skipped in safe mode.

db_security — data-leak hunting

Secrets, Connection Strings & Admin GUIs

What it really does & looks for: Beyond the live engines, it aggressively hunts for database credentials that leak through the website itself. It requests well-known secret/config files (.env, config/database.yml, my.cnf, wp-config.php, appsettings.json, docker-compose.yml…) and extracts DB credentials from them (passwords redacted in the report); it scans page and inline-JS source for hard-coded connection strings (mongodb://, postgres://, jdbc:…); it tests GraphQL endpoints for introspection (full schema disclosure); and it looks for exposed admin GUIs (phpMyAdmin, Adminer, Fauxton, Kibana…), downloadable SQL/SQLite dumps, and framework debug endpoints that print the DB config.

If it finds something — the attack: Any one of these can hand over the database directly — a readable .env or a leaked connection string is the password itself; an exposed admin GUI or dump is a one-click breach.

Good to know: Every credential shown is masked. A leaked secret found here should be treated as compromised and the password rotated immediately.

db_security — attack path & loot

Exploitation Plan & Extracted Data

What it really does & looks for: After the audit it assembles a red-team view of the result, surfaced in the console panel and the HTML report. It computes a DB Exposure Score (0-100 with a grade), prints an ordered Attack Path of copy-paste exploitation commands (the sqlmap line for each SQLi, redis-cli for Redis, curl for Elasticsearch/CouchDB/secret files…), and a Loot summary listing the data actually pulled during the scan (sample keys, index names, leaked secrets, default credentials).

If it finds something — the attack: This turns a list of findings into an actionable engagement plan: an operator can copy each command and reproduce the access, exactly as in a real assessment.

Good to know: Everything is opt-in and authorisation-gated — Hades shows the commands; it only runs sqlmap itself when you pass --exploit and confirm you are authorised.

5 · Scoring & Output

scorer

Security Score & Grade

What it really does & looks for: Turns all findings into a single 0-100 score and an A-F grade. Each finding subtracts points based on its severity, with diminishing returns per module (so one noisy module cannot sink the score) and an optional confidence weighting.

If it finds something — the attack: Not an attack — it is your at-a-glance health indicator. A low grade means several real, actionable issues were found.

> **Good to know:** INFO findings are context only and NEVER affect the grade — the score reflects genuine problems (Low and above) only.